LLM-assisted Data Systems for Efficient ML Pipeline Generation and Query Optimization

Saeed Fathollahzadeh



Dissertation Defense

August 21, 2026



Information Retrieval P. BAXENDALE, Editor

A Relational Model of Data for Large Shared Data Banks

E. F. CODD
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the internal representation of data is changed. Changes in data representation will often be needed as a result of changes in query, update, and growth activities. It is important that terminal activities not be hampered when changes are made in the representation of data.

Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, inadequacies of these models are discussed. A model based on n-ary relations, a normal form for data base relations, and the concept of a universal data sublanguage are introduced. In Section 2, certain operations on relations (other than logical inference) are discussed briefly. In Section 3, the problem of representing given relations by stored relations is discussed briefly with emphasis on the influence of alternative representations on performance. In Section 4, the impact of redundancy on consistency is discussed briefly. Finally, in Section 5, the network model, the hierarchic model, and the present model are compared briefly with respect to their relational properties and potentialities for providing data independence.

KEY WORDS AND PHRASES: data bank, data base, data structure, data organization, hierarchy of data, network of data, relations, derivability, redundancy, consistency, composition, join, retrieval language, predicate calculus, security, data integrity
CR CATEGORIES: 3.70, 3.73, 4.20, 4.22, 4.29

1. Relational Model and Normal Form

1.1. INTRODUCTION

This paper is concerned with the application of elementary relation theory to the problem of providing a model of data for users that may wish to interrogate and update the data, as well as for the designers of computer systems that store data that others will use. At present, most computer systems encountered in practice are fairly complex from the users' points of view. Design decisions have frequently been made in favor of convenience of implementation or performance rather than convenience of use. As a consequence, considerable knowledge about data representation remains built into application programs and terminal activities. Many of these programs would have to be drastically changed if certain changes were made in the data representation. Changes in data representation will often be needed as a result of changes in query, update, and growth activities. It is important that terminal activities not be hampered when changes are made in the representation of data.

Access Path Selection in a Relational Database Management System

P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie

Extensible/Rule Based Query Rewrite Optimization in Starburst

Hamid Pirahesh, Joseph M. Hellerstein*, Waqar Hasan†
IBM Almaden Research Center, San Jose, CA 95120, USA
pirahesh@almaden.ibm.com, joey@cs.berkeley.edu, hasan@hpl.hp.com

Abstract

This paper describes the Query Rewrite facility of the Starburst extensible database system, a novel phase of query optimization. We present a suite of rewrite rules used in Starburst to transform queries into equivalent queries for faster execution, and also describe the production rule engine which is used by Starburst to choose and execute these rules. Examples are provided demonstrating that three Query Rewrite transformations lead to query execution time improvements of orders of magnitude, suggesting that Query Rewrite in general — and these rewrite rules in particular — are an essential step in query optimization for modern database systems.

1 Introduction

In traditional database systems, query optimization typically consists of a single phase of processing in which access methods, join orders and join methods are chosen to provide an efficient plan for executing a user's declarative query. We refer to this phase as *plan optimization*. In this paper we present a distinct phase of query optimization, Query Rewrite, which has been implemented in the Starburst DBMS [HCL+90] to precede plan optimization in the processing of a query.

The goal of Query Rewrite is twofold:

1. *Make queries as declarative as possible:* In database languages such as SQL, it is often possible for a purely expressed query, though ostensibly declarative, to force typical plan optimizers into choosing sub-optimal execution plans. A major goal of Query Rewrite is the transformation of such "procedural" queries into equivalent but more declarative queries.
2. *Perform natural heuristics:* Certain heuristics can be performed in Query Rewrite and are generally accepted in the literature as heuristics are done in typical plan optimizers, they often can be applied in a more general way during Query Rewrite.

Although it is accepted doctrine that query languages should be declarative, we shall see in our examples that alternative but equivalent expressions of a query can have widely varying performance, often differing by orders of magnitude. It is therefore our conviction that Query Rewrite is an essential step in query optimization, since it further ensures that the expression of the query will be insignificant with respect to its performance.

1.1 Queries with Path Expressions

The goals of Query Rewrite explained above are even more significant in Object Oriented applications, which typically generate complicated queries with "path expressions" connecting various collections of objects [BTAM, LLOW91, LLPS91]. In such applications, both the *complexity of the logic* and the *volume of the data* are far greater than in traditional DBMS applications [HSR88]. As a result, query optimization becomes increasingly important.

The following is an example of a query involving path expressions, using the Object SQL syntax defined in [BTAV91]. This query is a small violation of an example presented in [BTAV91]. The example database contains records of patients. Medical records are set attributes of patients. All accesses to data are via methods. Given a patient's record, the medical records are returned by the function get_medical_records. The example retrieves male patients who have been diagnosed with malaria or smallpox prior to '10/10/89'. The *FROM* clause enumerates the patients and the *WHERE* clause restricts the patients to males and checks for the existence of a malaria or a smallpox prior to a given date.

```
SELECT DISTINCT P
FROM Patient p  IN Patient_Set
```

Efficient and Robust Automated Machine Learning

Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, Frank Hutter
Department of Computer Science, University of Freiburg, Germany
{feurerm, kleinaa, eggensperger, springj, mblum, fh}@cs.uni-freiburg.de

AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML

Patara Trirat, Wonyong Jeong, Sung Ju Hwang

Abstract

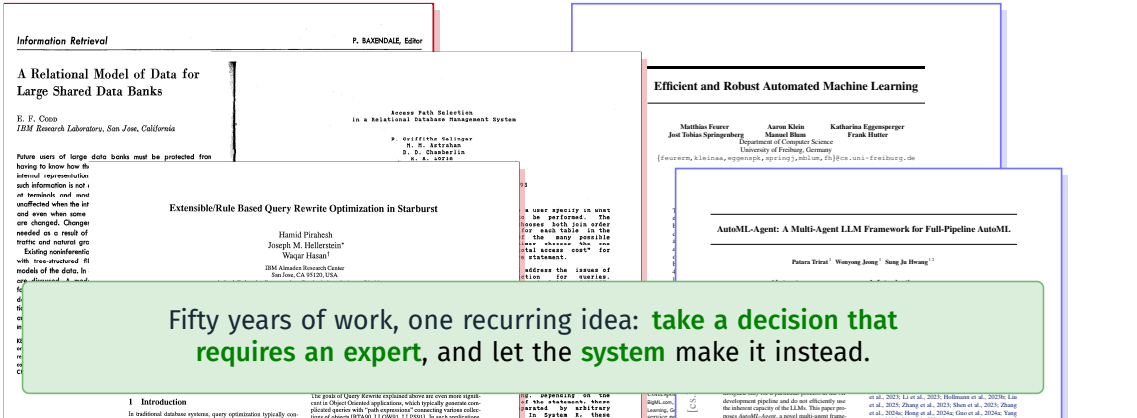
Automated machine learning (AutoML) accelerates AI development by automating tasks in the development pipeline, such as optimal model search and hyperparameter tuning. Existing AutoML systems often require technical expertise to set up complex tools, which is in general time-consuming and requires a large amount of human effort. Therefore, recent works have started exploiting large language models (LLM) to lessen such burden and increase the usability of AutoML frameworks via a natural language interface, allowing non-expert users to build their data-driven solutions. These methods, however, are usually designed only for a particular process in the AI development pipeline and do not efficiently use the inherent capacity of the LLMs. This paper proposes *AutoML-Agent*, a novel multi-agent framework tailored for full-pipeline AutoML, i.e., from data retrieval to model deployment. *AutoML-Agent* takes user's task descriptions, facilitates collaboration between specialized LLM agents, and delivers deployment-ready models. Unlike existing work, instead of devising a single plan, we introduce a retrieval-augmented planning strategy to enhance exploration to search for more optimal plans. We also decompose each plan into subtasks (e.g., data preprocessing and neural network design) each of which is solved by a specialized agent to build via prompting executing in parallel, making the search process more efficient. Moreover, we propose a multi-stage verification to verify executed results and guide the code generation

1. Introduction

Automated machine learning (AutoML) has significantly reduced the need for technical expertise and human labors in developing effective data-driven solutions by automating each process in the AI development pipeline (Thornton et al., 2013; Ren et al., 2020; He et al., 2021), such as feature engineering, model selection, and hyperparameter optimization (HPO). However, current AutoML systems (Gijsbers et al., 2024) often necessitate programming expertise to configure complex tools and resources, potentially creating barriers for a larger pool of users with limited skills and knowledge, such as domain experts (Sun et al. (2023); §A).

To make AutoML frameworks more accessible to non-expert users, many recent studies (Tirat et al., 2021; Viswanathan et al., 2023; Li et al., 2023; Hollmann et al., 2023b; Liu et al., 2025; Zhang et al., 2023; Shen et al., 2023; Zhang et al., 2024a; Hong et al., 2024a; Guo et al., 2024a; Yang et al., 2025; Chi et al., 2024) have suggested to use natural language interfaces with large language models (LLM) for machine learning (ML) and data science (DS) tasks. Nevertheless, these previous LLM-based AutoML frameworks only considered a limited number of tasks due to their restricted designs, either only for a process in the pipeline (e.g., feature engineering (Hollmann et al., 2023b; Li et al., 2024; Mulbery et al., 2024), HPO (Liu et al., 2024a; 2025; Zhang et al., 2024a), and model selection (Zhang et al., 2023; Shen et al., 2023)) or for a specific group of downstream tasks (e.g., natural language processing (Viswanathan et al., 2023) and computer vision (Yang et al., 2025)). In addition, most methods overlook the inherent capability of LLMs to search for promising solution models by leveraging training of the candidate models during the search process, making it prohibitively costly and slow.

arXiv:2410.02958v2 [cs.LG] 6 Jun 2025



Information Retrieval — P. BAXENDALE, Editor

A Relational Model of Data for Large Shared Data Banks

E. F. Codd
IBM Research Laboratory, San Jose, California

Access Path Selection in a Relational Database Management System

P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie

Extensible/Rule Based Query Rewrite Optimization in Starburst

Hamid Pirahesh, Joseph M. Hellerstein*, Waqar Hasan†
IBM Almaden Research Center, San Jose, CA 95120, USA

Efficient and Robust Automated Machine Learning

Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, Frank Hutter
Department of Computer Science, University of Freiburg, Germany
{feurerm,kleinaa,eggenspk,springj,mblum,fh}@cs.uni-freiburg.de

AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML

Patara Trirat, Wonyong Jeong, Sung Ju Hwang

Fifty years of work, one recurring idea: **take a decision that requires an expert**, and let the **system** make it instead.



















+ AutoML and LLM-based pipeline builders

auto-sklearn | AutoGluon | TPOT | H2O AutoML | CAAFE | AIDE | AutoML-Agent

Thousands of building blocks — but **choosing, cleaning, configuring and gluing** them is still **manual expert work**





















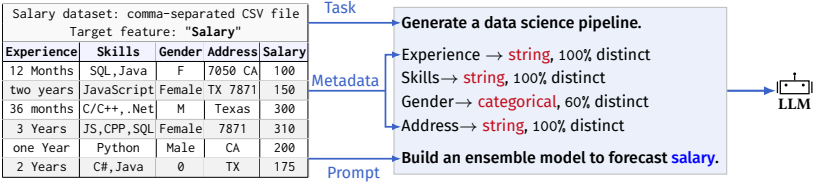

+ decades of rewrite rules and optimizer frameworks

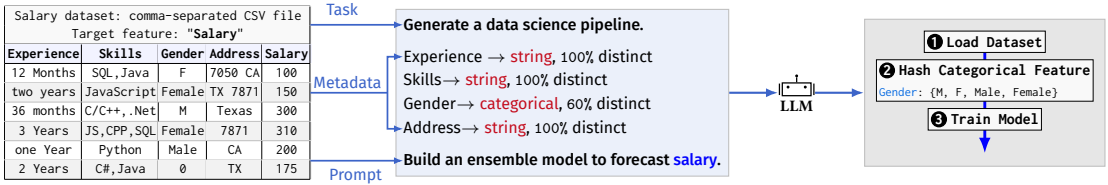
- System R
- Starburst
- Volcano/Cascades
- Apache Calcite
- LearnedRewrite
- R-Bot

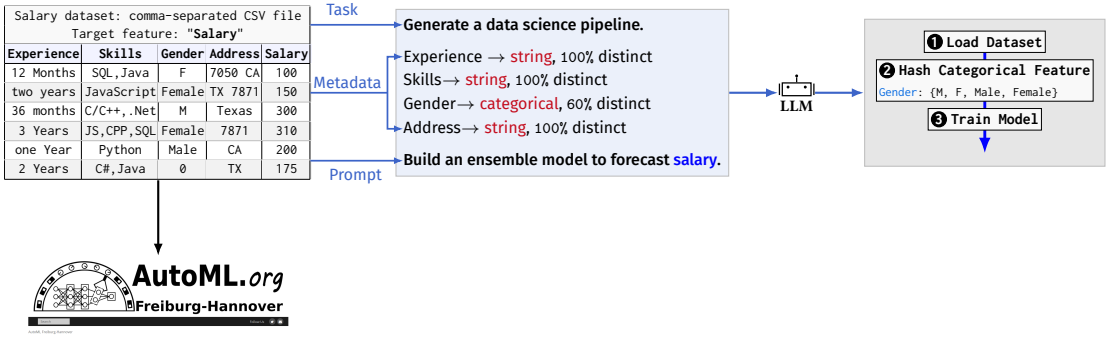
Same data, same semantics — runtime still varies by **orders of magnitude** with how the query is **written**

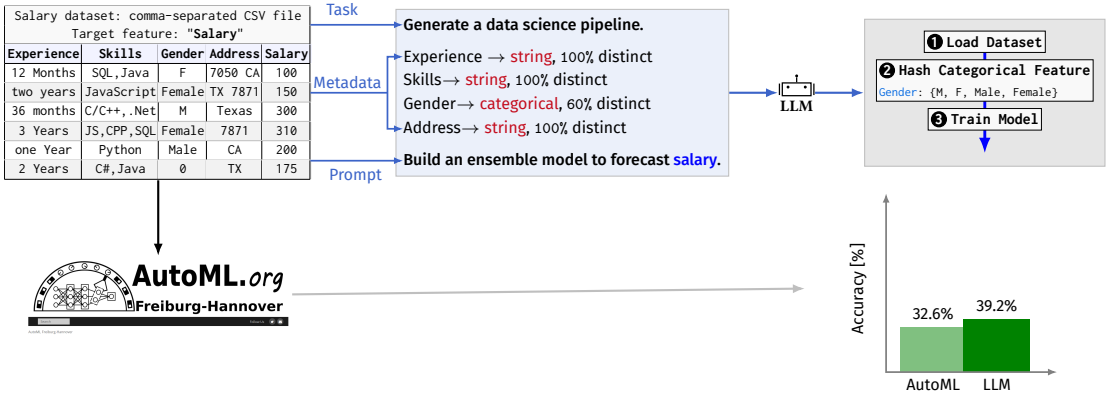
Salary dataset: comma-separated CSV file
Target feature: "**Salary**"

Experience	Skills	Gender	Address	Salary
12 Months	SQL,Java	F	7050 CA	100
two years	JavaScript	Female	TX 7871	150
36 months	C/C++,.Net	M	Texas	300
3 Years	JS,CPP,SQL	Female	7871	310
one Year	Python	Male	CA	200
2 Years	C#,Java	0	TX	175


Salary dataset: comma-separated CSV file
Target feature: "Salary"
Experience | Skills | Gender | Address | Salary
12 Months | SQL,Java | F | 7050 CA | 100
two years | JavaScript | Female | TX 7871 | 150
36 months | C/C++,.Net | M | Texas | 300
3 Years | JS,CPP,SQL | Female | 7871 | 310
one Year | Python | Male | CA | 200
2 Years | C#,Java | 0 | TX | 175
Task
Metadata
Prompt
Generate a data science pipeline.
Experience → string, 100% distinct
Skills → string, 100% distinct
Gender → categorical, 60% distinct
Address → string, 100% distinct
Build an ensemble model to forecast salary.
LLM


Salary dataset: comma-separated CSV file
Target feature: "Salary"
Experience | Skills | Gender | Address | Salary
12 Months | SQL,Java | F | 7050 CA | 100
two years | JavaScript | Female | TX 7871 | 150
36 months | C/C++,.Net | M | Texas | 300
3 Years | JS,CPP,SQL | Female | 7871 | 310
one Year | Python | Male | CA | 200
2 Years | C#,Java | 0 | TX | 175
Task
Metadata
Prompt
Generate a data science pipeline.
Experience → string, 100% distinct
Skills → string, 100% distinct
Gender → categorical, 60% distinct
Address → string, 100% distinct
Build an ensemble model to forecast salary.
LLM
1 Load Dataset
2 Hash Categorical Feature
Gender: {M, F, Male, Female}
3 Train Model


Salary dataset: comma-separated CSV file
Target feature: "Salary"
Experience | Skills | Gender | Address | Salary
12 Months | SQL,Java | F | 7050 CA | 100
two years | JavaScript | Female | TX 7871 | 150
36 months | C/C++,.Net | M | Texas | 300
3 Years | JS,CPP,SQL | Female | 7871 | 310
one Year | Python | Male | CA | 200
2 Years | C#,Java | 0 | TX | 175
Task
Metadata
Prompt
Generate a data science pipeline.
Experience → string, 100% distinct
Skills → string, 100% distinct
Gender → categorical, 60% distinct
Address → string, 100% distinct
Build an ensemble model to forecast salary.
LLM
1 Load Dataset
2 Hash Categorical Feature
Gender: {M, F, Male, Female}
3 Train Model
AutoML.org
Freiburg-Hannover


Salary dataset: comma-separated CSV file
Target feature: "Salary"
Experience | Skills | Gender | Address | Salary
12 Months | SQL,Java | F | 7050 CA | 100
two years | JavaScript | Female | TX 7871 | 150
36 months | C/C++,.Net | M | Texas | 300
3 Years | JS,CPP,SQL | Female | 7871 | 310
one Year | Python | Male | CA | 200
2 Years | C#,Java | 0 | TX | 175
Task
Metadata
Prompt
Generate a data science pipeline.
Experience → string, 100% distinct
Skills → string, 100% distinct
Gender → categorical, 60% distinct
Address → string, 100% distinct
Build an ensemble model to forecast salary.
LLM
1 Load Dataset
2 Hash Categorical Feature
Gender: {M, F, Male, Female}
3 Train Model
AutoML.org
Freiburg-Hannover
Accuracy [%]
32.6%
39.2%
AutoML
LLM

- SQL is **declarative**: the user states *what* to compute, the optimizer decides *how*.

```
SELECT COUNT(*)
FROM PostHistory h,
     Posts p,
     Users u,
     Badges b
WHERE b.UserId = u.Id
     AND p.OwnerUserId = u.Id
     AND h.UserId = u.Id
     AND h.CreationDate >= '2010-07-27 18:08:19' ::timestamp
     AND h.CreationDate <= '2014-09-10 08:22:43' ::timestamp
     AND p.PostTypeId = 2;
```

- SQL is **declarative**: the user states *what* to compute, the optimizer decides *how*.

```
1: SELECT COUNT(*)------  No early aggregation ⇒ huge intermediates
2: FROM PostHistory h,
3:      Posts p,
4:      Users u,
5:      Badges b
6: WHERE b.UserId = u.Id
7:      AND p.OwnerUserId = u.Id
8:      AND h.UserId = u.Id
9:      AND h.CreationDate >= '2010-07-27 18:08:19' ::timestamp
10:     AND h.CreationDate <= '2014-09-10 08:22:43' ::timestamp
11:     AND p.PostTypeId = 2;
```

- SQL is **declarative**: the user states *what* to compute, the optimizer decides *how*.

```
1: SELECT COUNT(*)
2: FROM PostHistory h,
3:      Posts p,
4:      Users u,
5:      Badges b
6: WHERE b.UserId = u.Id
7:      AND p.OwnerUserId = u.Id
8:      AND h.UserId = u.Id
9:      AND h.CreationDate >= '2010-07-27 18:08:19' ::timestamp
10:     AND h.CreationDate <= '2014-09-10 08:22:43' ::timestamp
11:     AND p.PostTypeId = 2;
```

No early aggregation ⇒ huge intermediates

Flat 4-way Cross Join

- SQL is **declarative**: the user states *what* to compute, the optimizer decides *how*.

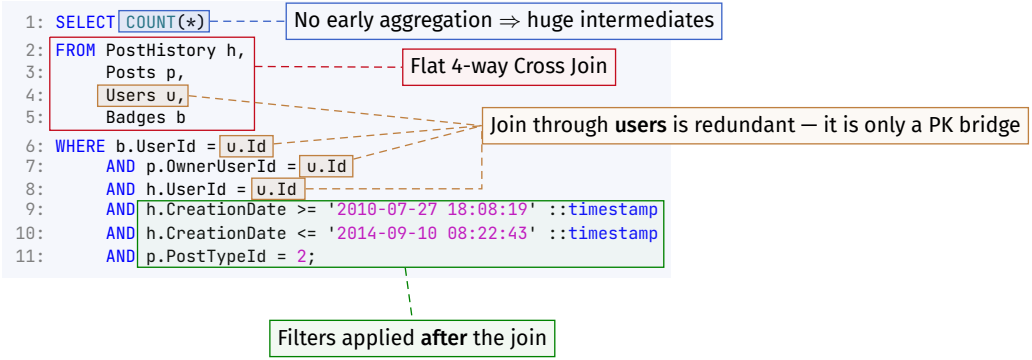
```
1: SELECT COUNT(*)
2: FROM PostHistory h,
3:      Posts p,
4:      Users u,
5:      Badges b
6: WHERE b.UserId = u.Id
7:       AND p.OwnerUserId = u.Id
8:       AND h.UserId = u.Id
9:       AND h.CreationDate >= '2010-07-27 18:08:19' ::timestamp
10:      AND h.CreationDate <= '2014-09-10 08:22:43' ::timestamp
11:      AND p.PostTypeId = 2;
```

No early aggregation ⇒ huge intermediates

Flat 4-way Cross Join

Join through **users** is redundant — it is only a PK bridge

- SQL is **declarative**: the user states *what* to compute, the optimizer decides *how*.



- SQL is **declarative**: the user states *what* to compute, the optimizer decides *how*.

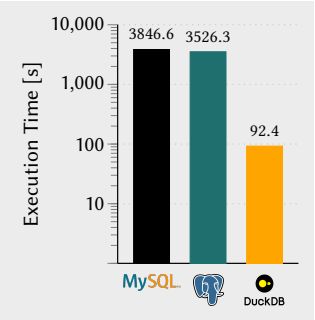
```
1:  SELECT COUNT(*)
2:  FROM PostHistory h,
3:       Posts p,
4:       Users u,
5:       Badges b
6:  WHERE b.UserId = u.Id
7:        AND p.OwnerUserId = u.Id
8:        AND h.UserId = u.Id
9:        AND h.CreationDate >= '2010-07-27 18:08:19' ::timestamp
10:       AND h.CreationDate <= '2014-09-10 08:22:43' ::timestamp
11:       AND p.PostTypeId = 2;
```

No early aggregation ⇒ huge intermediates

Flat 4-way Cross Join

Join through **users** is redundant — it is only a PK bridge

Filters applied **after** the join



- SQL is **declarative**: the user states *what* to compute, the optimizer decides *how*.

```
1: SELECT COUNT(*)
2: FROM PostHistory h,
3:      Posts p,
4:      Users u,
5:      Badges b
6: WHERE b.UserId = u.Id
7:       AND p.OwnerUserId = u.Id
8:       AND h.UserId = u.Id
9:       AND h.CreationDate >= '2010-07-27 18:08:19' ::timestamp
10:      AN
11:      AN
```

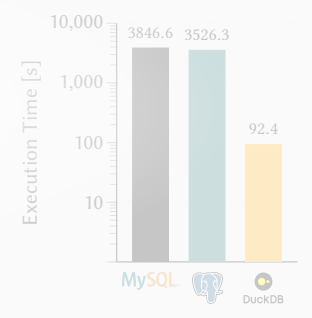
No early aggregation ⇒ huge intermediates

Flat 4-way Cross Join

Join through **users** is redundant — it is only a PK bridge

The optimizer is not going to save you here

Filters applied **after** the join



ML Pipeline Generation

- Data prep dominates the effort
- LLMs hallucinate on **unseen** data
- AutoML searches blindly & slowly

SQL Query Optimization

- Runtime swings with query **form**
- Hand-written rules are brittle
- Rewrites must stay **equivalent**

ML Pipeline Generation

- Data prep dominates the effort
- LLMs hallucinate on **unseen** data
- AutoML searches blindly & slowly

SQL Query Optimization

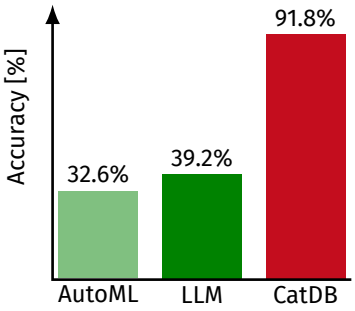
- Runtime swings with query **form**
- Hand-written rules are brittle
- Rewrites must stay **equivalent**

Research vision: make LLMs a **reliable** component of data systems — **generate** like an expert, **verify** like a system.

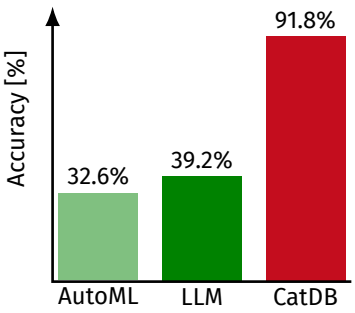
Key idea: pair the **creativity of LLMs** with the **verification power of data systems**
generate candidates → verify them → keep only what is **correct** and **fast**.

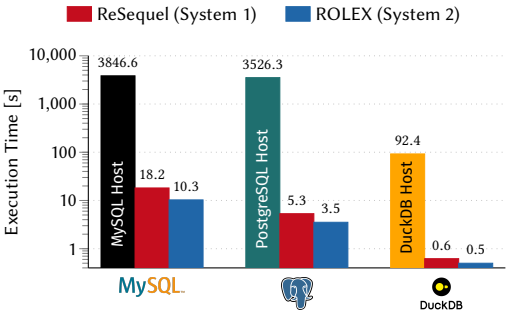
Three systems, one recipe:

- **CatDB**: a **data catalog** grounds the prompt; pipelines are **executed and auto-repaired** before they are accepted
- **ReSequel**: **query templates** steer the rewrite; **sample-based checking** verifies equivalence before execution
- **ROLEX**: rewrite **rules** are discovered from the **workload**, verified once, and then reused across queries



✅ The data-centric ML pipeline generated by **CatDB** improves model performance by 52.6% on the Salary dataset (Example 1).





✅ The data-centric ML pipeline generated by **CatDB** improves model performance by 52.6% on the Salary dataset (Example 1).

✅ **ReSequel** rewrites the query: 148–662× faster on DuckDB, MySQL, and PostgreSQL (Example 2).

✅ **ROLEX** rewrites the query: 162–998× faster.

Overview | Thesis Contributions

CatDB + GIO

From raw data to trained models

PVLDB '25

CatDB: Data-catalog-guided, LLM-based Generation of Data-centric ML Pipelines

SIGMOD '25 Demo

Demonstrating CatDB: LLM-based Generation of Data-centric ML Pipelines

SIGMOD '23

GIO: Generating Efficient Matrix and Frame Readers for Custom Data Formats by Example

ReSequel

An LLM rewrite layer over any DBMS

PVLDB '26

ReSequel: Robust LLM-assisted Query Rewriting and Optimization using Templatization and Sampling

- Rewrite once per template
- Sample-verified rewrites
- **2.5–16**× per workload

ROLEX

Distilling those rewrites into rules

SIGMOD '27 — under submission

Workload-optimized Rewrite Rule Discovery and Generation

Saeed Fathollahzadeh, Concordia University, Canada, saeed.fathollahzadeh@concordia.ca
Essam Mansour, Concordia University, Canada, essam.mansour@concordia.ca
Matthias Boehm, Technische Universität Berlin, Germany, matthias.boehm@tu-berlin.de

- Offline LLM → C++ rules
- Zero LLM cost at runtime
- **60%** below ReSequel

CatDB GIO ReSequel



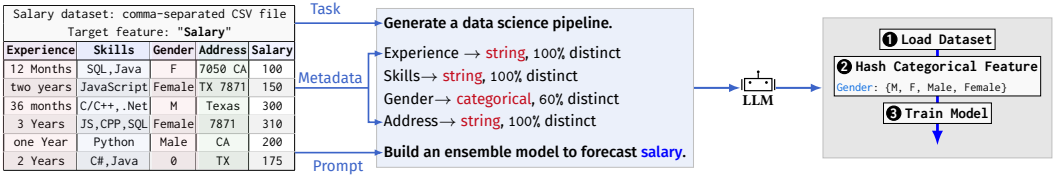


CatDB: Data-catalog-guided, LLM-based Generation of Data-centric ML Pipelines

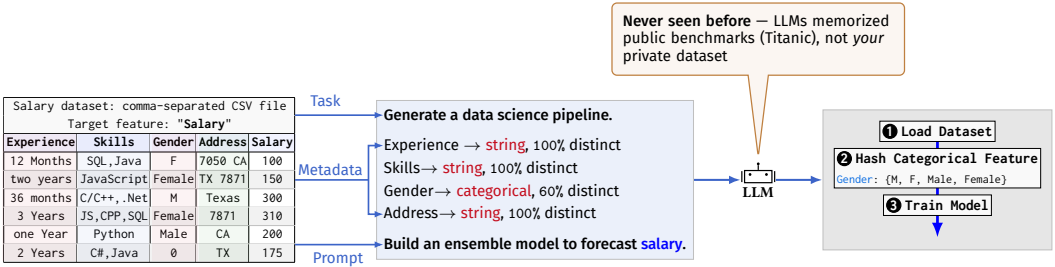
PVLDB '25
SIGMOD '25 (Demo)

Saeed Fathollahzadeh
Essam Mansour
Matthias Boehm

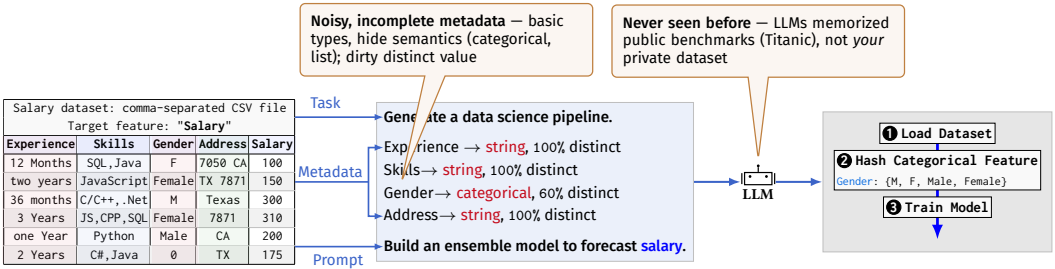
CatDB | Motivation



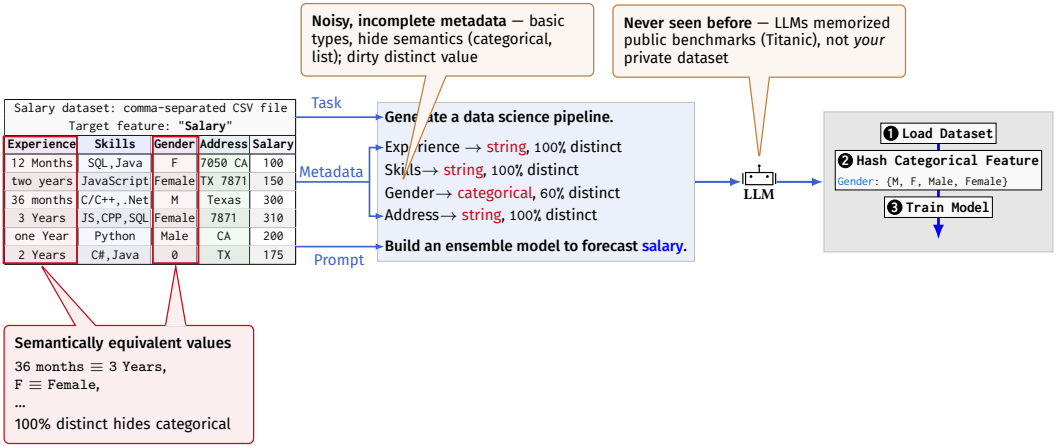
CatDB | Motivation



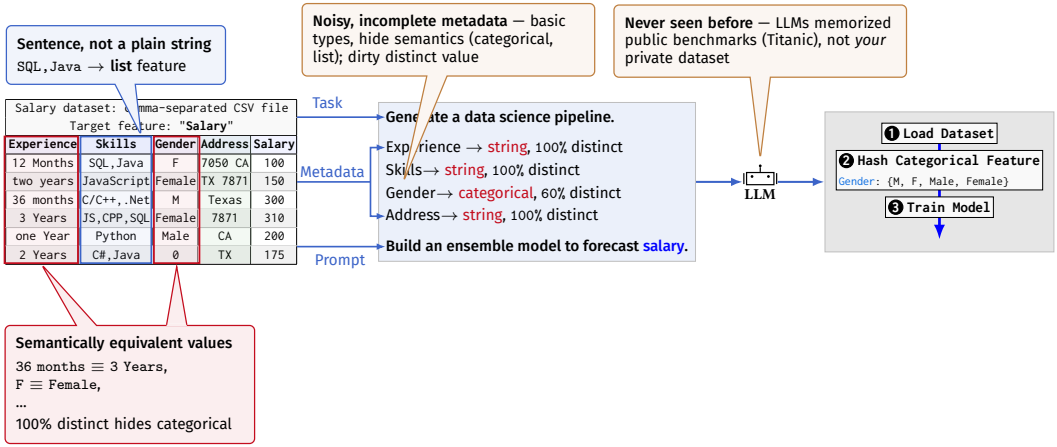
CatDB | Motivation



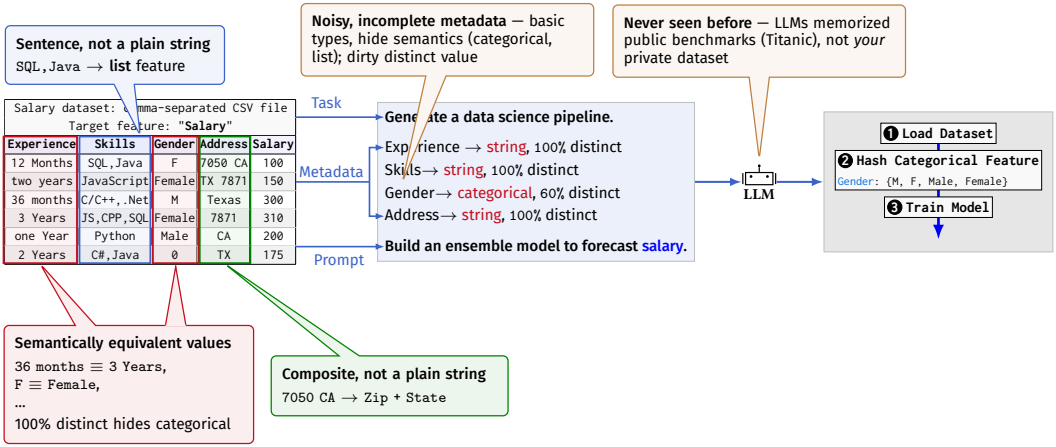
CatDB | Motivation



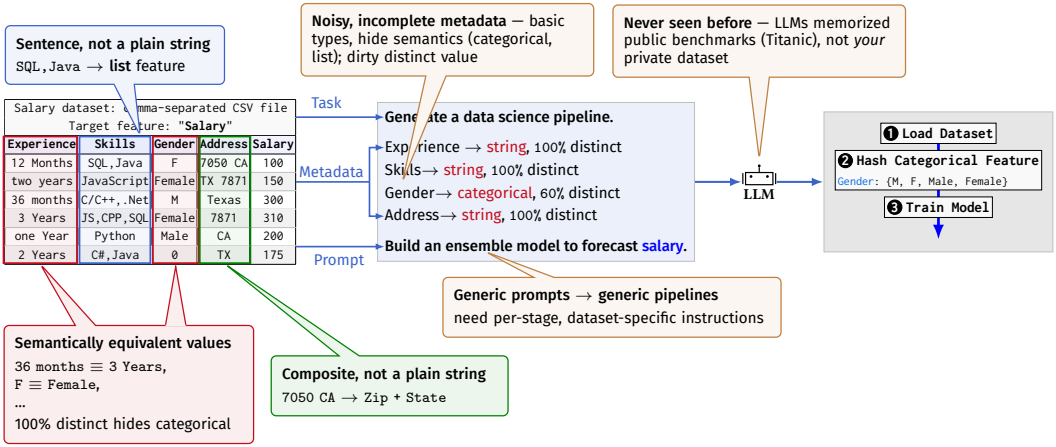
CatDB | Motivation



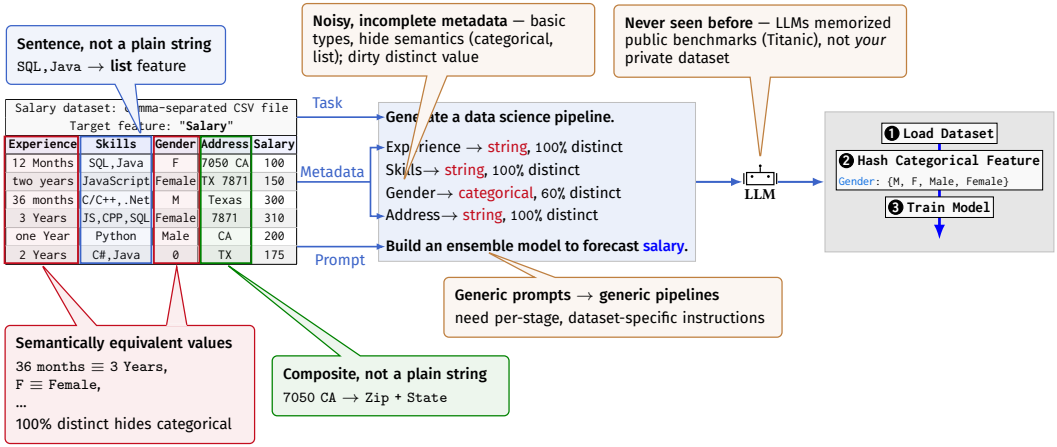
CatDB | Motivation



CatDB | Motivation

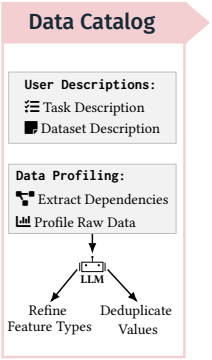


CatDB | Motivation

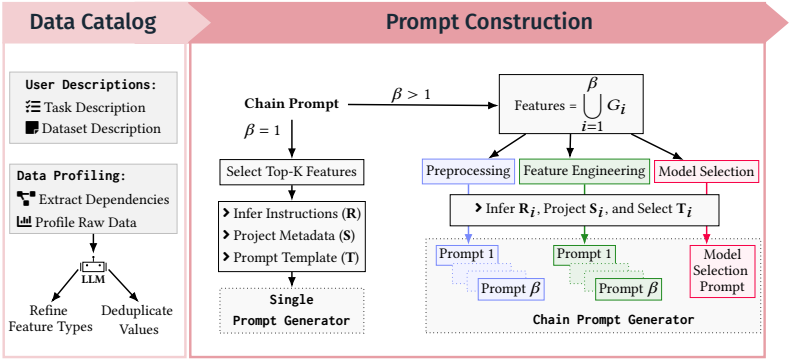


Data catalogs store profiling info that can **precisely guide** LLMs

- **Zero-shot in-context learning** — no task-specific examples, no fine-tuning: the data catalog guides the LLM, and execution verifies it.

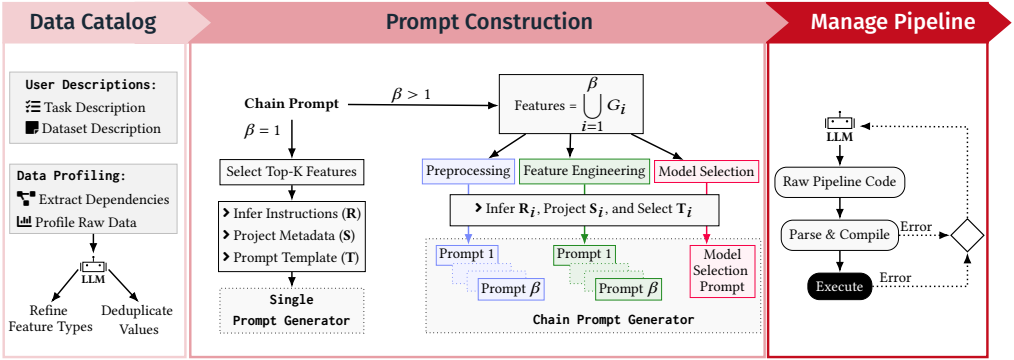


- **Zero-shot in-context learning** — no task-specific examples, no fine-tuning: the data catalog guides the LLM, and execution verifies it.



CatDB | LLM-based Generation of Data-centric ML Pipelines

- **Zero-shot in-context learning** — no task-specific examples, no fine-tuning: the data catalog guides the LLM, and execution verifies it.



CatDB | Schema Refinement

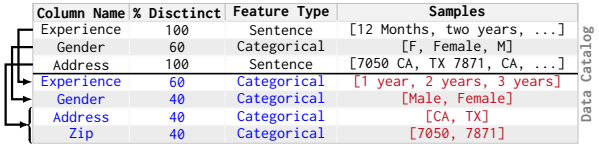
- **Problems**
  - **#1 Different representations** for the same categorical value
  - **#2 Concatenated lists** / composite features

Raw Dataset

#	Experience	Gender	Address	Salary
1	12 Months	F	7050 CA	100
2	two years	Female	TX 7871	150
3	36 months	M	Texas	300
4	3 Years	Female	7871	310
5	one Year	M	CA	200

Clean Dataset

#	Experience	Gender	Address	Zip	Salary
1	1 year	Female	CA	7050	100
2	2 years	Female	TX	7871	150
3	3 years	Male	TX		300
4	3 years	Female		7871	310
5	1 year	Male	CA		200



CatDB | Schema Refinement

- **Problems**
  - **#1 Different representations** for the same categorical value
  - **#2 Concatenated lists** / composite features

Raw Dataset

#	Experience	Gender	Address	Salary
1	12 Months	F	7050 CA	100
2	two years	Female	TX 7871	150
3	36 months	M	Texas	300
4	3 Years	Female	7871	310
5	one Year	M	CA	200

Clean Dataset

#	Experience	Gender	Address	Zip	Salary
1	1 year	Female	CA	7050	100
2	2 years	Female	TX	7871	150
3	3 years	Male	TX		300
4	3 years	Female		7871	310
5	1 year	Male	CA		200

Data Catalog

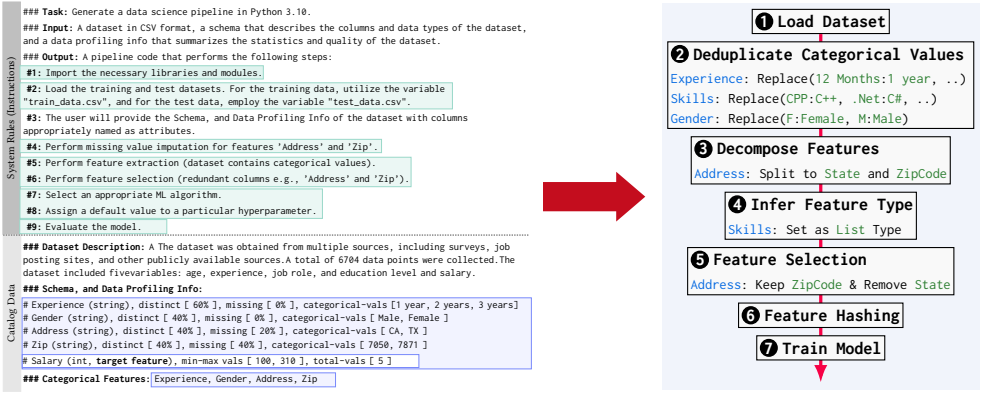
Column Name	% Disctinct	Feature Type	Samples
Experience	100	Sentence	[12 Months, two years, ...]
Gender	60	Categorical	[F, Female, M]
Address	100	Sentence	[7050 CA, TX 7871, CA, ...]
Experience	60	Categorical	[1 year, 2 years, 3 years]
Gender	40	Categorical	[Male, Female]
Address	40	Categorical	[CA, TX]
Zip	40	Categorical	[7050, 7871]

- **Approach**
  - Identify candidate **string** features
  - Build a prompt from the feature name and its distinct values
  - Let the LLM (GPT, Gemini, Llama) create a mapping from the original to a reduced value set, and infers the feature type → **Iterative refinement**

Method	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	#11	#12	#13	#14
Original	563	256	148	64	119	53	59	46	24	11	48	12	15	4
CatDB	**100**	43	45	32	95	39	47	36	16	3	43	6	9	3

CatDB | Prompt Construction and Pipeline Management

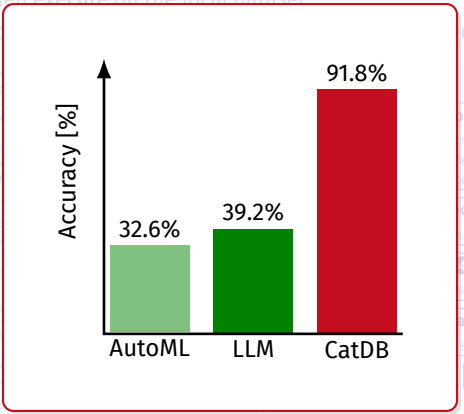
- **Rules (R) + schema (S)** in template **T**: coding task per stage + refined catalog
- **Validate**: syntax check, then **execute** on the local dataset
- **Repair loop**: knowledge-base patches; syntax/runtime errors re-prompted with the catalog slice



CatDB-generated prompt (instructions + refined metadata) for the **Salary** dataset, and the resulting pipeline

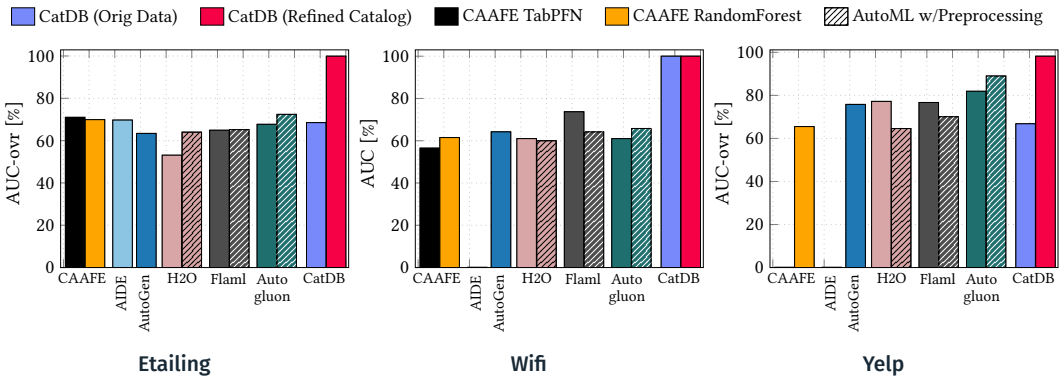
CatDB | Prompt Construction and Pipeline Management

- **Rules (R) + schema (S)** in template **T**: coding task per stage + refined catalog
- **Validate**: syntax check, then execute on the local dataset
- **Repair loop**: knowledge-b… …d with the catalog slice



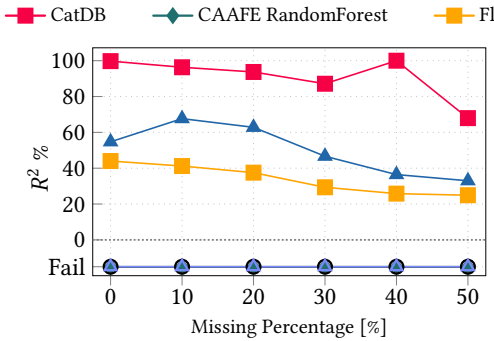
CatDB-generated prompt (instructions + refined metadata) for the **Salary** dataset, and the resulting pipeline

CatDB | Performance Comparison (LLM = Gemini-1.5)

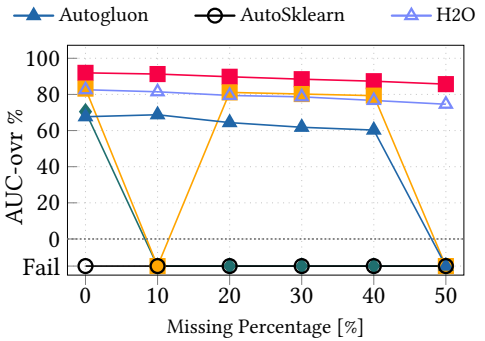


- **Deduplication ➜** Removes duplicates, balances labels.
- **Categorical Features ➜** Fixes formatting, transforms complexity.
- **Feature Refinement ➜** Drops constant/misread features, preserves distribution.

CatDB | Outlier and Missing Value Injection (Gemini-1.5)



(a) **Utility MV** + **Outlier** = 5%



(b) **Volkert MV** + **Outlier** = 5%

■ **Robustness** ➜ CatDB maintains high performance even with increasing missing values and 5% outliers.

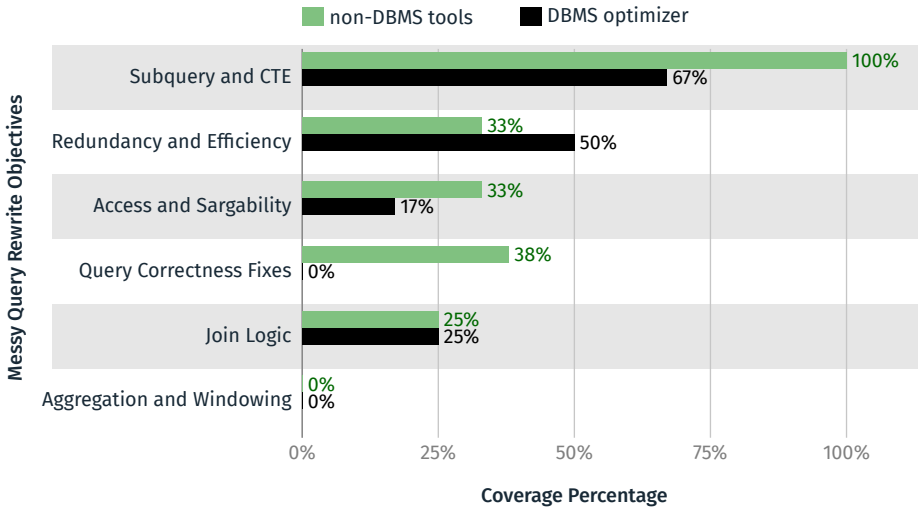



ReSequel: Robust LLM-assisted Query Rewriting and Optimization using Templatization and Sampling

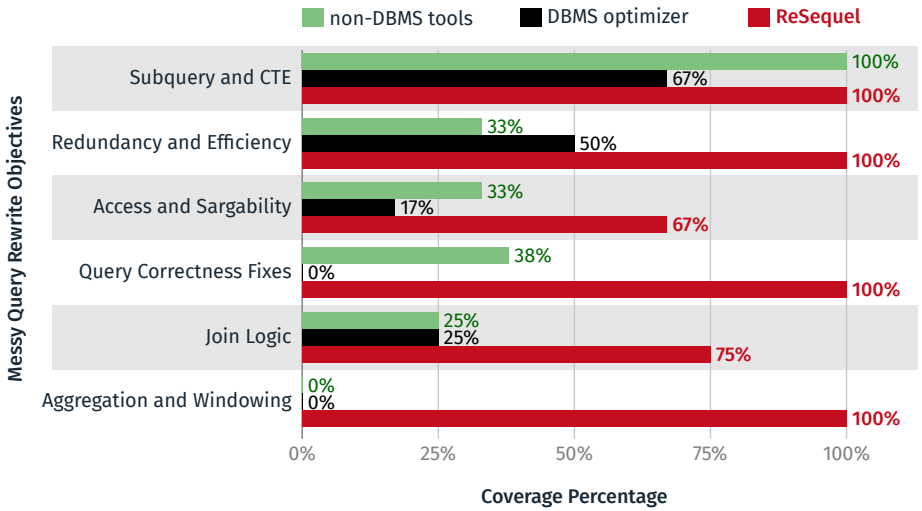
PVLDB '26

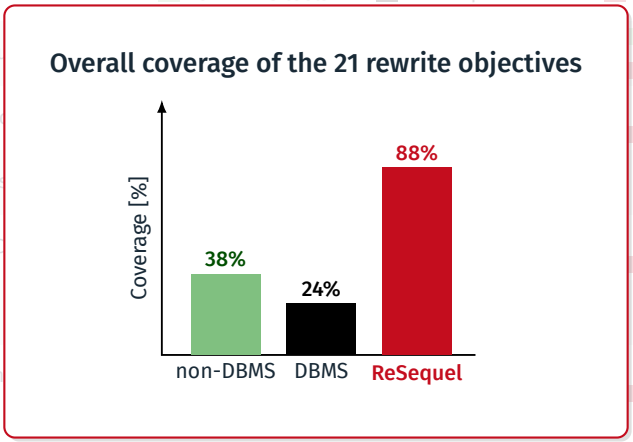
Saeed Fathollahzadeh
Essam Mansour
Matthias Boehm

ReSequel | Motivation — What Do Query Rewriters Actually Cover?

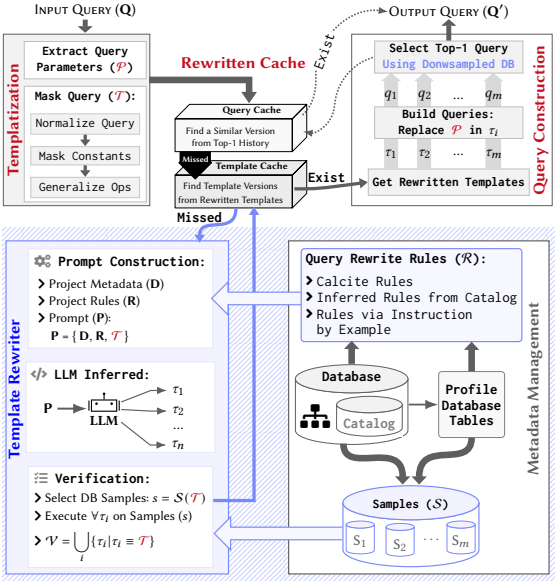


ReSequel | Motivation — What Do Query Rewriters Actually Cover?




Overall coverage of the 21 rewrite objectives
Coverage [%]
38%
24%
88%
non-DBMS
DBMS
ReSequel

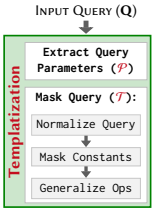
ReSequel | LLM Rewriting as an Outer Optimization Layer



Key Design Decisions:

- Outer layer: any DBMS
- Template-based: rewrite once
- Metadata-guided: schema + stats
- Sample-verified: correctness check
- Cached: amortized over workload

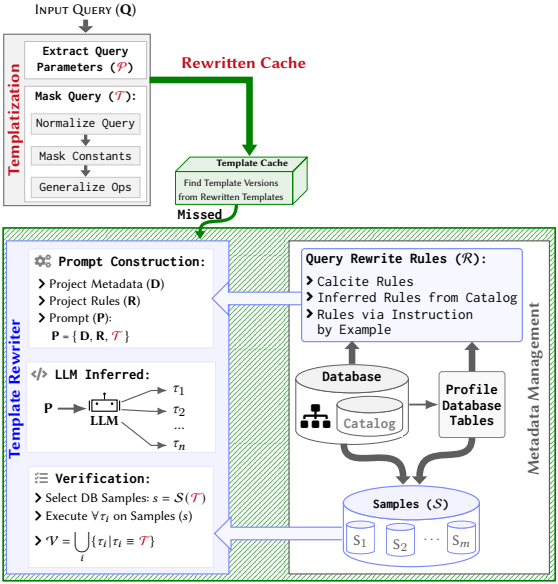
ReSequel | LLM Rewriting as an Outer Optimization Layer



Templatization:

- **Normalize:** canonical form
- **Mask constants:** literals $\rightarrow$ placeholders
- **Generalize ops:** same shape, one key
- Output: template $\mathcal{T}$ + params $\mathbf{P}$
- Effect: many queries, one rewrite

ReSequel | LLM Rewriting as an Outer Optimization Layer

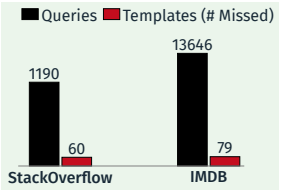


Metadata Management:

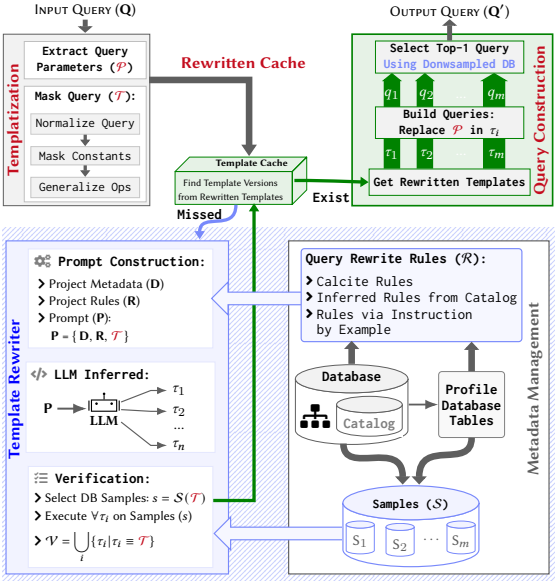
- **Catalog:** schema, keys, constraints
- **Profiling:** per-table/column statistics
- **Rules** $\mathcal{R}$: Calcite, catalog, by-example
- **Samples** $\mathcal{S}$: small, representative

Template Rewriter:

- **Prompt** $\mathbf{P}$: metadata + rules
- **LLM proposes** N candidate rewrites
- Verify on samples, not on full DB
- Output: verified set $\mathcal{V}$ (**equivalent only**)



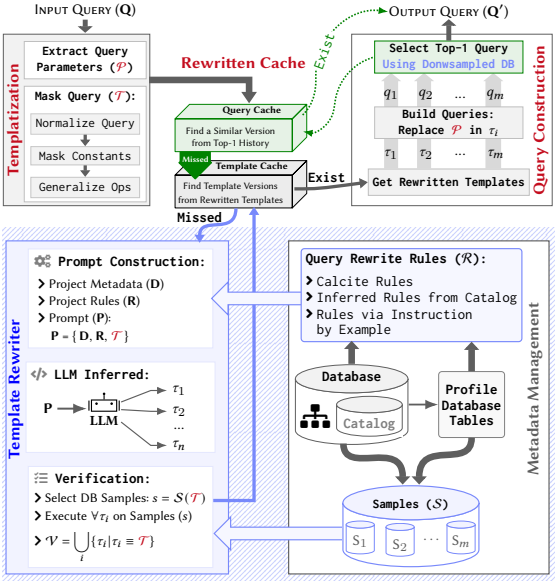
ReSequel | LLM Rewriting as an Outer Optimization Layer



Query Construction:

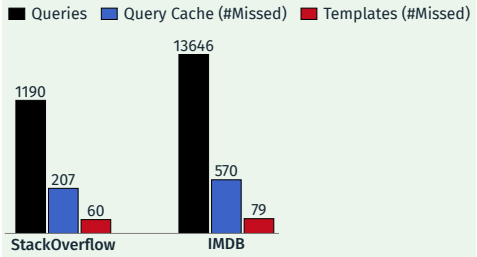
- Fetch verified templates $\tau_1, \ldots, \tau_m$
- **Bind params:** replace $\mathbf{P}$ in each τ_i
- **Rank candidates** on downsampled DB
- **Select Top-1** query $\mathbf{Q}'$
- DBMS executes $\mathbf{Q}'$ unchanged

ReSequel | LLM Rewriting as an Outer Optimization Layer



Query Cache:

- **Lookup:** similar query in Top-1 history
- **Hit:** reuse the cached Top-1 rewrite
- **No LLM call**, no sample runs, no ranking
- Miss: fall back to the template cache
- Cost amortized over the workload



ReSequel | Recap — Example 2: Declarative Query Processing

- SQL is **declarative**: the user states *what* to compute, the optimizer decides *how*.

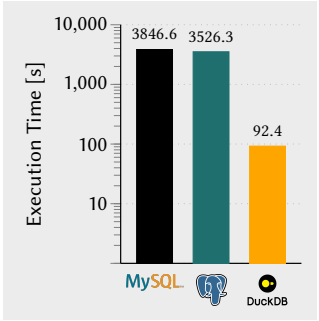
```
1: SELECT COUNT(*)
2: FROM PostHistory h,
3:      Posts p,
4:      Users u,
5:      Badges b
6: WHERE b.UserId = u.Id
7:      AND p.OwnerUserId = u.Id
8:      AND h.UserId = u.Id
9:      AND h.CreationDate >= '2010-07-27 18:08:19' ::timestamp
10:     AND h.CreationDate <= '2014-09-10 08:22:43' ::timestamp
11:     AND p.PostTypeId = 2;
```

No early aggregation ⇒ huge intermediates

Flat 4-way Cross Join

Join through **users** is redundant — it is only a PK bridge

Filters applied **after** the join



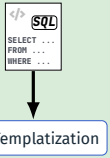
ReSequel | An Example Workflow

Qyery Template (Q122, Stats): ❶

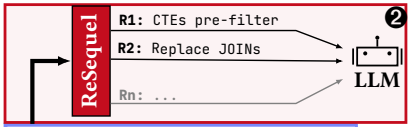
```
 1:SELECT COUNT(*)
 2:FROM History h,
 3:  Posts p,
 4:  Users u,
 5:  Badges b
 6:WHERE b.UserId = u.Id
 7:  AND p.OwnerUserId = u.Id
 8:  AND h.UserId = u.Id
 9:  AND h.CreationDate >= '$$$'
10:  AND h.CreationDate <= '$$$'
11:  AND p.PostTypeId = ###;
```

Masked Predicates

Rewriting pipeline:



ReSequel | An Example Workflow

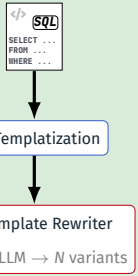


Qyery Template (Q122, Stats): ❶

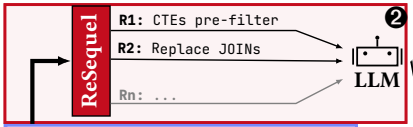
```
 1:SELECT COUNT(*)
 2:FROM History h,
 3:  Posts p,
 4:  Users u,
 5:  Badges b
 6:WHERE b.UserId = u.Id
 7:  AND p.OwnerUserId = u.Id
 8:  AND h.UserId = u.Id
 9:  AND h.CreationDate >= '$$$'
10:  AND h.CreationDate <= '$$$'
11:  AND p.PostTypeId = ###;
```

Masked Predicates

Rewriting pipeline:



ReSequel | An Example Workflow



Qyery Template (Q122, Stats): ❶

```
1:SELECT COUNT(*)
2:FROM History h,
3:  Posts p,
4:  Users u,
5:  Badges b
6:WHERE b.UserId = u.Id
7:  AND p.OwnerUserId = u.Id
8:  AND h.UserId = u.Id
9:  AND h.CreationDate >= '$$$'
10: AND h.CreationDate <= '$$$'
11: AND p.PostTypeId = ###;
```

Masked Predicates

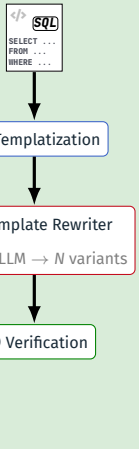
Output Template (Version #1):
Output Template (Version #2):
Output Template (Version #3):
Output Template (Version #4):
Output Template (Version #5):
Output Template (Version #6):

Output Template (Version #7):

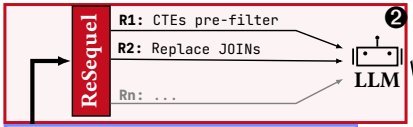
```
1:WITH FilteredHistory AS (
2:  SELECT UserId, COUNT(*) h_count
3:  FROM History WHERE CreationDate >= '$$$'
4:    AND CreationDate <= '$$$' GROUP BY UserId),
5:FilteredPosts AS (
6:  SELECT OwnerUserId, COUNT(*) p_count
7:  FROM Posts WHERE PostTypeId = ###
8:  GROUP BY OwnerUserId),
9:FilteredBadges AS (
10:  SELECT UserId, COUNT(*) b_count
11:  FROM Badges GROUP BY UserId)
12:SELECT SUM(ph.ph_count * p.p_count * b.b_count)
13:FROM Users u
14:  JOIN FilteredHistory h ON u.Id = h.UserId
15:  JOIN FilteredPosts p ON u.Id = p.OwnerUserId
16:  JOIN FilteredBadges b ON u.Id = b.UserId;
```

❸ Downsample DB (10%)
Verify result equivalence

Rewriting pipeline:



ReSequel | An Example Workflow



Qyery Template (Q122, Stats): ❶

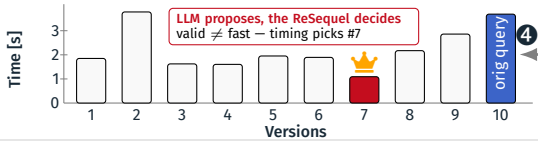
```
1:SELECT COUNT(*)
2:FROM History h,
3:  Posts p,
4:  Users u,
5:  Badges b
6:WHERE b.UserId = u.Id
7:  AND p.OwnerUserId = u.Id
8:  AND h.UserId = u.Id
9:  AND h.CreationDate >= '$$$'
10: AND h.CreationDate <= '$$$'
11: AND p.PostTypeId = ###;
```

Masked Predicates

Output Template (Version #1):
Output Template (Version #2):
Output Template (Version #3):
Output Template (Version #4):
Output Template (Version #5):
Output Template (Version #6):

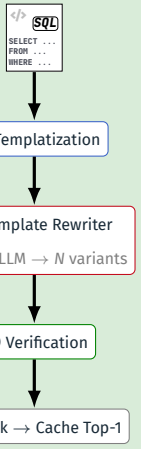
Output Template (Version #7):

```
1:WITH FilteredHistory AS (
2:  SELECT UserId, COUNT(*) h_count
3:  FROM History WHERE CreationDate >= '$$$'
4:    AND CreationDate <= '$$$' GROUP BY UserId),
5:FilteredPosts AS (
6:  SELECT OwnerUserId, COUNT(*) p_count
7:  FROM Posts WHERE PostTypeId = ###
8:  GROUP BY OwnerUserId),
9:FilteredBadges AS (
10:  SELECT UserId, COUNT(*) b_count
11:  FROM Badges GROUP BY UserId)
12:SELECT SUM(ph.ph_count * p.p_count * b.b_count)
13:FROM Users u
14:  JOIN FilteredHistory h ON u.Id = h.UserId
15:  JOIN FilteredPosts p ON u.Id = p.OwnerUserId
16:  JOIN FilteredBadges b ON u.Id = b.UserId;
```



Downsample DB (10%)
Verify result equivalence ❸

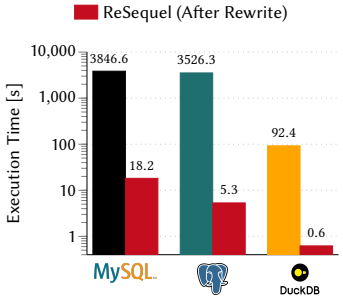
Rewriting pipeline:



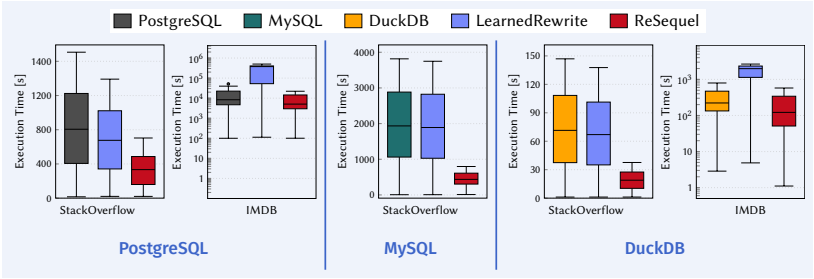
What ReSequel did to Q122

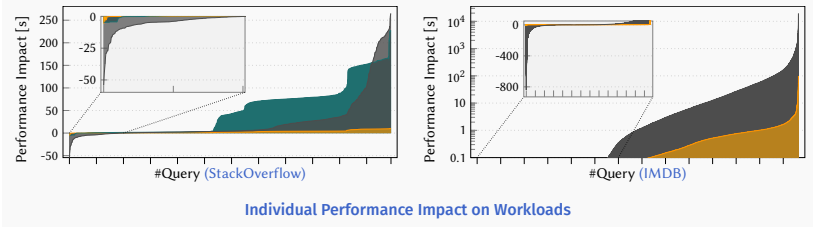
- **Factorized join** (eager aggregation) — `COUNT(*)` becomes a **product of per-key counts**; the flat join is never materialized
- **No 4-way cross join** — explicit joins on the CTEs; `Users` was only a PK bridge
- **Filters pushed into the CTEs** — applied *before* any join

148×–662× faster, same result



ReSequel | End-to-End Performance on Large Workloads





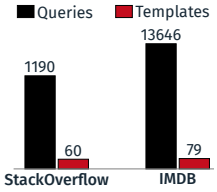
Individual Performance Impact on Workloads

Highlights:

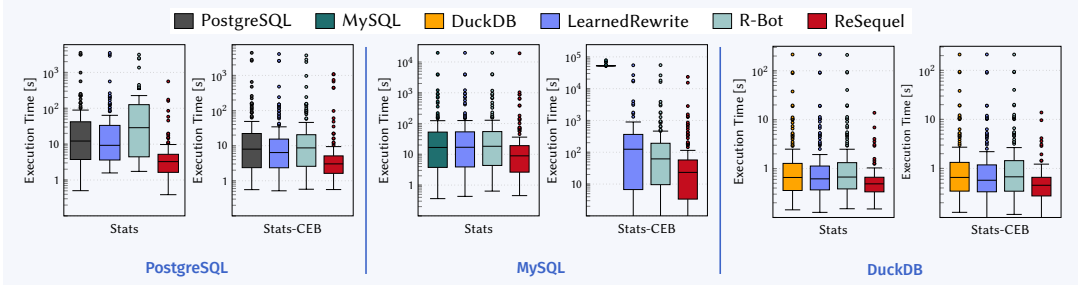
Workload speed-up

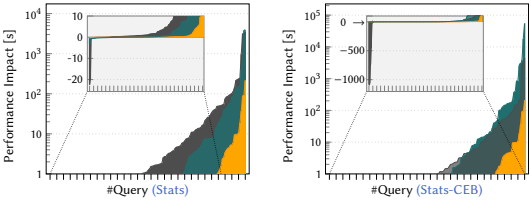
Host DBMS	StackOv.	IMDB
PostgreSQL	2.4×	2.3×
MySQL	4.8×	—
DuckDB	3.9×	1.4×

- **Every host DBMS improves** — not a single-engine effect
- **LLM baselines cannot run** — R-Bot, LLM-R2 rewrite *per query*
- **Per query:** most speed up



ReSequel | End-to-End Performance on Small Workloads





Individual Performance Impact on Workloads

Highlights:

Workload speed-up:

Workload	PostgreSQL	MySQL	DuckDB
Stats	5.89×	1×	14.85×
Stats-CEB	4×	3.3×	16×

- **Where the gain comes from**: up to **20 variants** per query, *data*/*catalog* split, redundant `GROUP BY` removed
- **R-Bot**: rewrites only **12%** of the workload — with **negative** impact
- **LearnedRewrite**: per-query, Calcite-bound — improves few queries, **often slowdowns**

ROLEX: Workload-optimized Rewrite Rule Discovery and Generation

SIGMOD '27

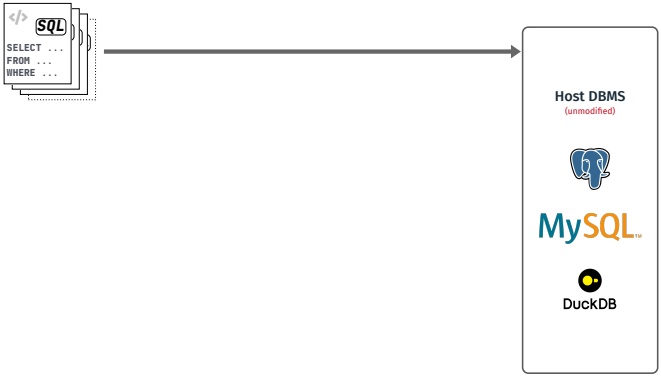
(under submission)

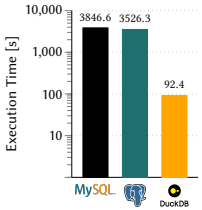
Saeed Fathollahzadeh

Essam Mansour

Matthias Boehm

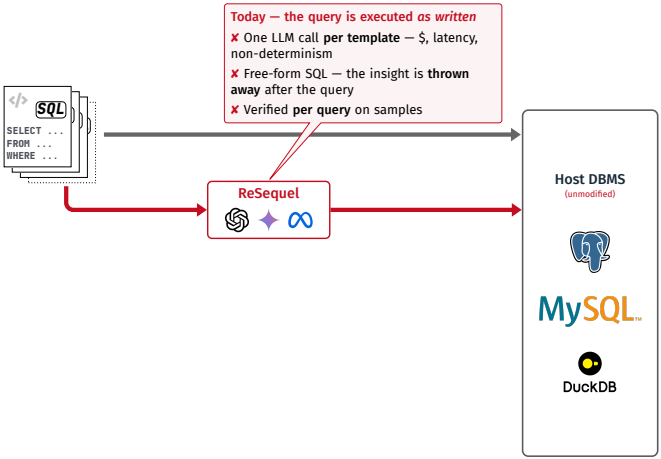
ROLEX | Motivation — Where Should the LLM Live?

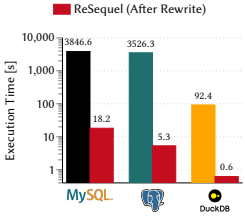




Example 2 — Stats-CEB query 122

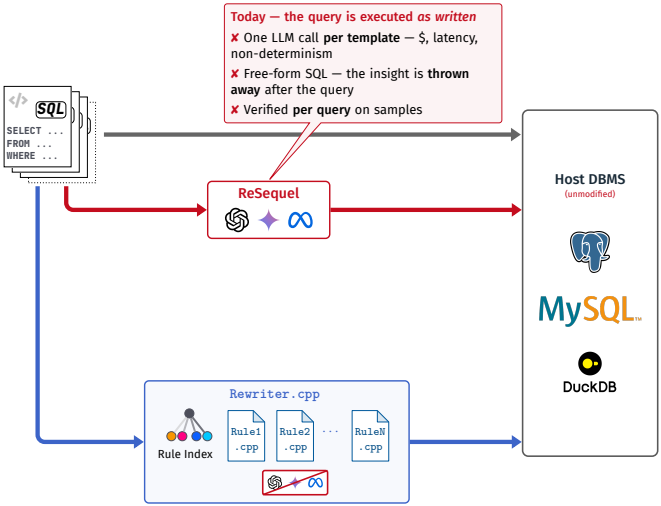
ROLEX | Motivation — Where Should the LLM Live?

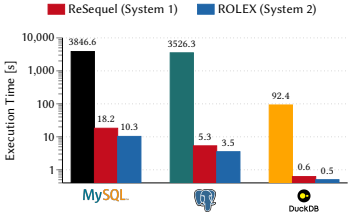




Example 2 — Stats-CEB query 122

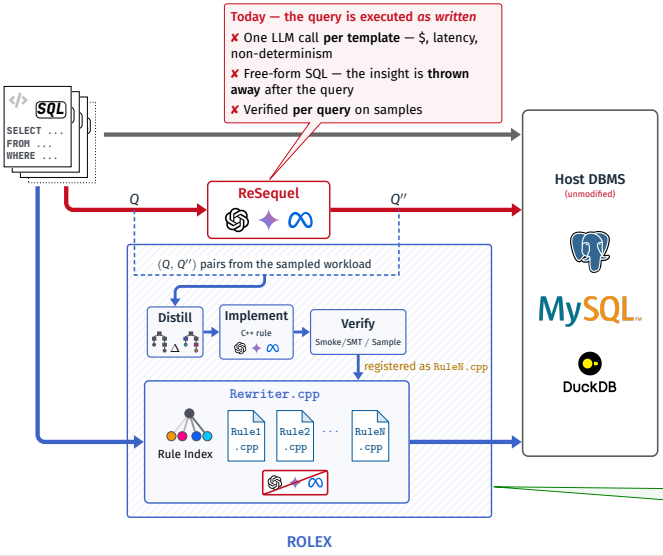
ROLEX | Motivation — Where Should the LLM Live?

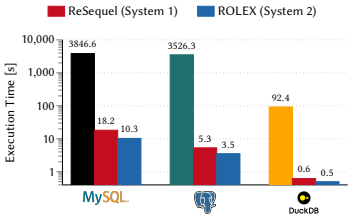




Example 2 — Stats-CEB query 122

ROLEX | Motivation — Where Should the LLM Live?



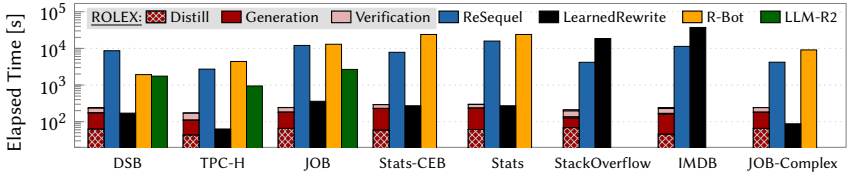


Example 2 — Stats-CEB query 122

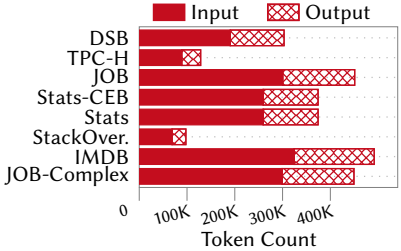
With ROLEX — rules

- ✔ **Reproducible** — same rewrite every run
- ✔ **Reusable** — one rule, many queries
- ✔ **Cheap** — zero online LLM cost
- ✔ **Verified** — Smoke / SMT / sample, once

ROLEX | What the Rules Cost to Build — Paid Once, Offline



(a) Rule-generation time per workload — the baselines pay their cost *per query*, at runtime

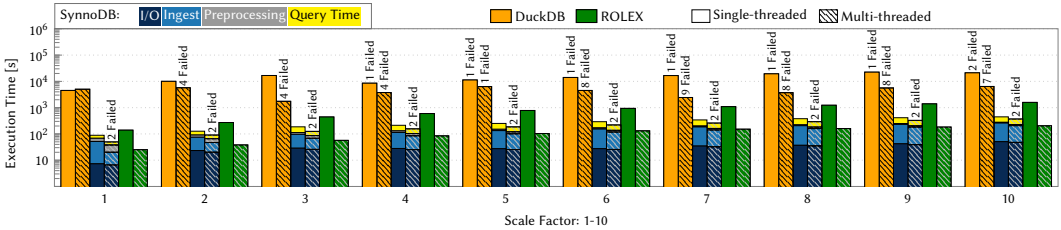


(b) LLM tokens per workload

Highlights:

- **Paid once, then free** — $\sim$2–5 $\times 10^2$ s per workload; R-Bot and LLM-R2 call the LLM **for every query**
- **Distill** $\rightarrow$ **Generate** $\rightarrow$ **Verify**, all in memory — the cost is **DBMS-independent**
- **118 rules** $\rightarrow$ **60 distinct** after cross-workload dedup; tokens track only the *new* rules
- **LLM-R2** also needs **per-workload re-training**

ROLEX | vs. a SynnoDB (BespokeOLAP) Query Compiler — CEB Workload at Scale



16 CEB queries over IMDB, SF 1–10 — SynnoDB compiles each query to native code over Parquet

Why ROLEX wins:

- Attacks the **asymptotic cause**: factorized `COUNT(*)` keeps intermediates under 50 GB — nothing spills
- 4 queries are **93%** of DuckDB's time → 378×, 220×, 102×, 34×; 32 threads now *help* (5.5–7.8×)
- **All 16 complete**, every SF — end-to-end 1.97×/1.81× over SynnoDB

Where the baselines break:

- **SynnoDB** executes fastest — but I/O + ingest + preprocessing are **58–77%** of wall clock
- **SynnoDB** multi-threaded **crashes** (`SIGSEGV`) on 11a/11b at *every* scale factor
- **DuckDB** spills ∼800 GB against 148 GB RAM: 32 threads are **slower than 1**, 4–9 queries time out

One recipe, three systems: generate with an LLM, **verify with the system**, keep only what is **correct** and **fast**

System	What grounds the LLM	What verifies it	Headline result
CatDB	data catalog + refined schema	execute & auto-repair	up to 52% accuracy
ReSequel	query templates + metadata	sample-based equivalence	2.5–16× per workload
ROLEX	the *workload* itself	SMT + sample, once, offline	60% below ReSequel, **zero** LLM online

Lessons learned:

- **Show the model the real data** — the catalog, the schema, the real queries; it cannot guess them
- **Always check the answer** — a pipeline or a rewrite that looks right can still be wrong
- **Ask the LLM once, not per query** — keep what it found, and every later query is free

Thank You!

One recipe, three systems: generate with an LLM, **verify with the system**, keep only what is **correct** and **fast**

System	What grounds the LLM	What verifies it	Headline result
CatDB	data catalog + refined schema	execute & auto-repair	up to 52% accuracy
ReSequel	query templates + metadata	sample-based equivalence	2.5–16× per workload
ROLEX	the *workload* itself	SMT + sample, once, offline	60% below ReSequel, **zero** LLM online

Lessons learned:

- **Show the model the real data** — the catalog, the schema, the real queries; it cannot guess them
- **Always check the answer** — a pipeline or a rewrite that looks right can still be wrong
- **Ask the LLM once, not per query** — keep what it found, and every later query is free

Appendix

1 CatDB

2 ReSequel

3 ROLEX

Future Work | Four Open Problems

Verifying generated ML pipelines

the catalog made *generation* reliable — checking what the operators actually *do* is still open

From a sample to a mini database

a small sample keeps results right, but loses cardinality — we need a miniature DB that keeps both

Equivalence for all of SQL

generating rules is the easy half; proving a rewrite equivalent, over every operator, is the bottleneck

A lightweight LLM for databases

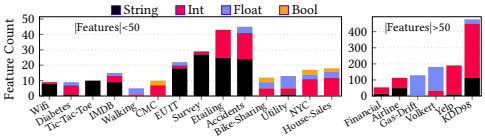
not every semantic rewrite fits a local rule — the model stays in the loop, so keep it small

Verification is the bottleneck: the better we can check, the more we can let the model do

CatDB | Datasets and Data Characteristics

ID	Dataset	#Tables	*n* (nrow)	*m* (ncol)	Type	#Classes
1	Wifi	1	98	9	Binary	2
2	Diabetes	1	768	9	Binary	2
3	Tic-Tac-Toe	1	958	10	Binary	2
4	IMDB	7	30,530,313	15	Binary	2
5	KDD98	1	82,318	478	Binary	2
6	Walking	1	149,332	5	Multiclass	22
7	CMC	1	1,473	10	Multiclass	3
8	EU IT	1	1,253	23	Multiclass	148
9	Survey	1	2,778	29	Multiclass	9
10	Etailing	1	439	44	Multiclass	5
11	Accidents	3	954,036	46	Multiclass	6
12	Financial	8	552,017	62	Multiclass	4
13	Airline	19	445,827	115	Multiclass	3
14	Gas-Drift	1	13,910	129	Multiclass	6
15	Volkert	1	58,310	181	Multiclass	10
16	Yelp	4	229,907	194	Multiclass	9
17	Bike-Sharing	1	17,379	12	Regression	869
18	Utility	1	4,574	13	Regression	95
19	NYC	1	581,835	17	Regression	1,811
20	House-Sales	1	21,613	18	Regression	4,028

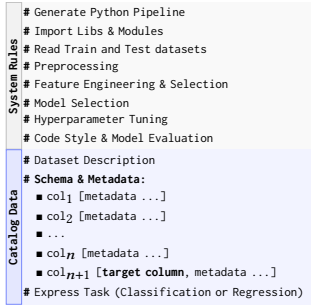
(a) Used datasets and their data characteristics



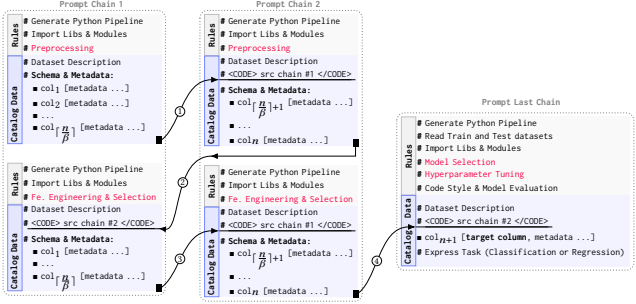
(b) Data type distribution

- **20 real-world datasets** — binary, multi-class, and regression
- **Single- and multi-table** — 1 to 19 tables, joined during preparation
- **Wide range** — 98 to 30,530,313 rows, 5 to 478 columns
- **Mixed feature types** — string, int, float, and bool in every task type
- **70/30** train/test split throughout

CatDB | How Is the Prompt Built — and Why Chain It?



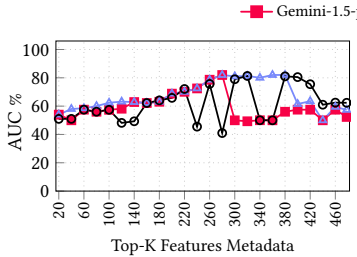
CatDB, $\beta = 1$



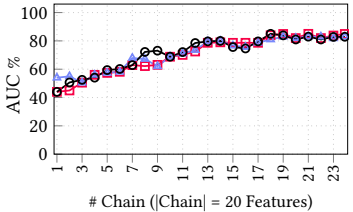
CatDB Chain, $\beta = 2$

- **Every prompt is two blocks** — projected **schema & metadata** from the data catalog, plus **rules** that spell out the LLM's task
- $\beta = 1$ — all columns and all rules in one prompt; fine until the dataset outgrows the LLM's token limit
- $\beta > 1$ **(chain)** — columns split into β chunks: all pre-processing prompts, then feature engineering, then *one* model-selection prompt
- **Each chain appends the previous code** (`<CODE>src chain #i</CODE>`) — keeps cross-chunk column dependencies and verifies each step alone, at the cost of repeated context

CatDB | Metadata Impact on Pipeline Performance



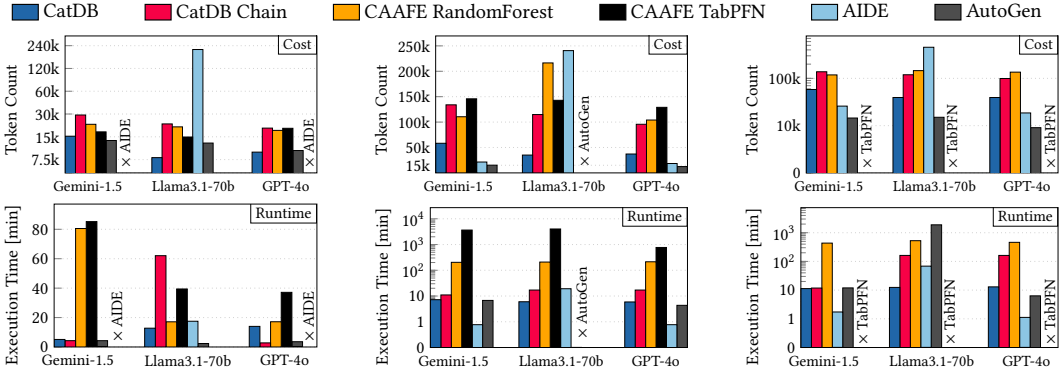
(a) KDD98 w/ CatDB prompts



(b) KDD98 w/ CatDB Chain prompts

- **More metadata is not better** — plain schema often beats schema plus statistics; excess metadata makes the LLM ignore the rules
- **Single prompt breaks down** past ≈ **260 features** — rules get dropped and AUC swings wildly (a)
- **Chaining fixes it** — 20 features per chain climbs **smoothly to ≈ 85% AUC** and holds, for all three LLMs (b)

CatDB | Cost and Runtime Comparison (10 iterations)



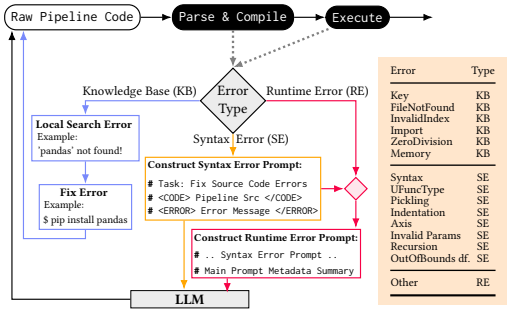
■ Cost Efficiency:

- AIDE & AutoGen use hand-crafted prompts
- CatDB consumes fewer tokens by projecting the Data Catalog

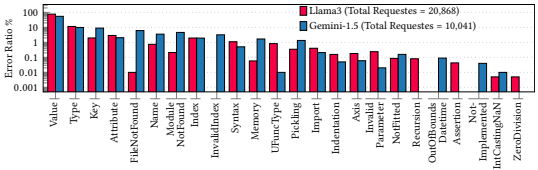
■ Runtime Speedup:

- CatDB is **8**×–**14**× faster
- AIDE & AutoGen consider only execution time

CatDB | Validation and Pipeline Error Management



(a) Error management & classification of error types



(b) Ratio and distribution of the 23 error types

LLM	Total Req.	KB [%]	SE [%]	RE [%]
Llama3.1-70b	20,868	2.464	2.907	94.629
Gemini-1.5 pro	10,041	21.213	2.092	76.695

(c) Error distributions of the error-trace dataset

CatDB | Relation to KramaBench

KramaBench (2025): 104 real data-to-insight pipelines over data lakes, 6 scientific domains

Common ground: raw data in, executable Python out, judged by *running* it — and both find that an unguided LLM fails, so both add structure (DS-GURU decomposes the task, CatDB projects catalog metadata + rules)

	KramaBench	CatDB
Input	Data lake of heterogeneous raw files — *which* files matter is unknown	One dataset (1–19 tables), target column named
Task	NL question → insight, split into canonical subtasks: discover → load → clean → analyse → answer	Supervised ML pipeline: pre-process → feature engineering → model selection → trained model
Oracle	Human reference solution per pipeline and per subtask	**None needed** — the held-out labels are the oracle
Verify	Compare produced output against the reference output, scored per subtask and end-to-end	Parse + execute on sample data, then accuracy on the validation split
On failure	Scored as a miss — the benchmark *measures*, it does not repair	KB lookup → LLM repair prompt → re-execute; **no silent errors**

Complementary: KramaBench names the failure modes — **discovery and cleaning** — that CatDB's catalog and verification loop attack

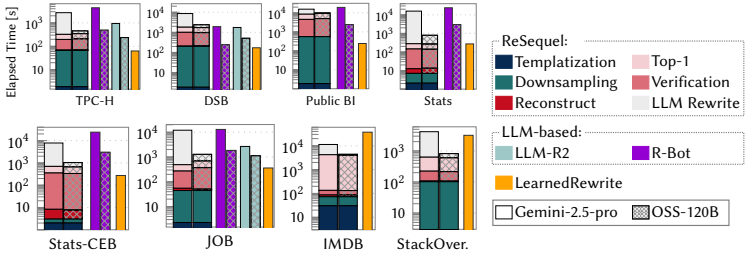
ReSequel | Workloads and Datasets

Dataset	#Tables	Workload	#Queries	#Templates
TPC-H	8	TPC-H (sf=10)	22	22
DSB	25	DSB (sf=100)	52	52
Public BI	83		100	100
Stats/Stats-CEB	8		146	146
JOB	21		113	95
StackOverflow	8	SQLStorm	1,192	60
IMDB	21		13,646	79

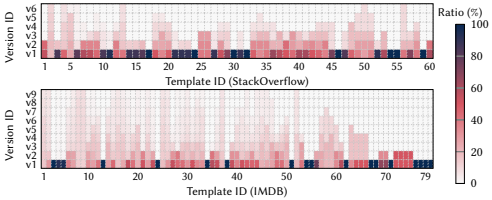
Used datasets and their workload characteristics

- **Group 1 — one template per query**
  TPC-H, DSB, Public BI, Stats/Stats-CEB: queries differ only in masked parameters, so templates = queries
- **Group 2 — partial overlap**
  JOB: ≈ 15% of queries share a template (113 queries, **95 templates**)
- **Group 3 — heavy overlap**
  StackOverflow and IMDB: 1,192 and 13,646 queries collapse to **60 and 79 templates**
- **Why the groups matter** — rewriting once *per template* only pays off when queries repeat; Group 3 is where that leverage is largest
- **Three hosts** — every workload runs on PostgreSQL, MySQL, and DuckDB

ReSequel | What Does Rewriting Cost?



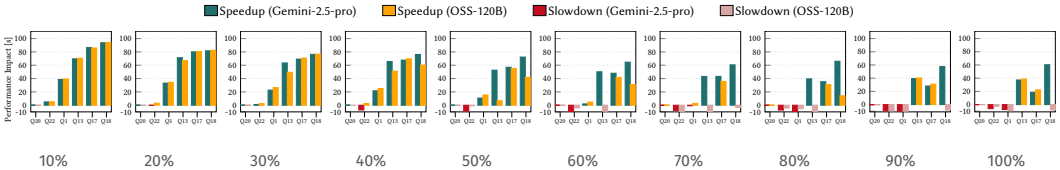
(a) Time breakdown of rewriting across 8 datasets (10% downsampling, PostgreSQL)



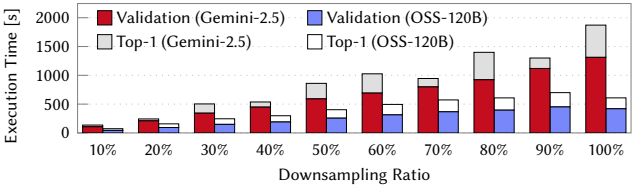
(b) Template and query cache hits

- **Templatization is free** — the catalog build dominates, not the LLM
- **LLM time scales with #templates**, not #queries (Gemini > OSS-120B, thinking mode)
- **Groups 1 & 2** — one template per query, so the cache never pays
- **Group 3** — **60 / 79** templates reused across 1,192 / 13,646 queries

ReSequel | How Much Data Does Verification Need?

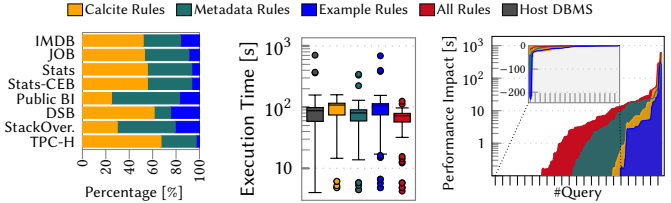


(a) Individual query performance impact at downsampling ratios 10%–100% (TPC-H SF10, PostgreSQL)



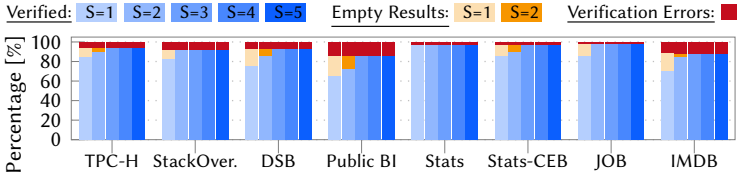
(b) Validation and top-1 selection overhead across downsampling ratios (TPC-H SF10, PostgreSQL)

ReSequel | Ablation Study



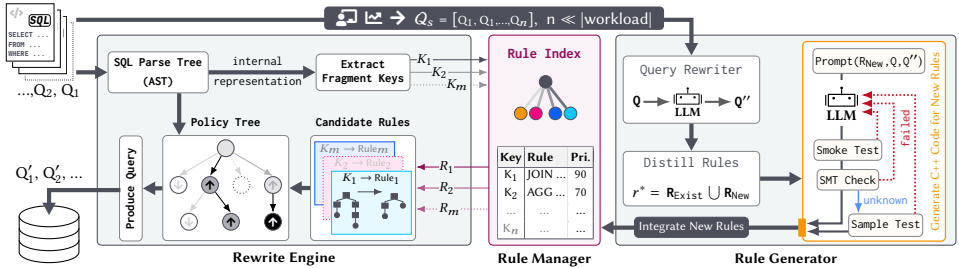
(a) Rule ratio · Overall (JOB) · Individual (JOB) — PostgreSQL

- **S=2–3** verifies every workload — more samples buy nothing
- All rules: **5.64×** on JOB
  Calcite **4.4×**, metadata **2×**
- Each category alone is erratic — the win is their overlap



(b) Verification rates across sample sizes (S=1–5; Gemini-2.5-pro, PostgreSQL)

ROLEX | Full System Architecture



Offline **Rule Generator** distills and verifies rules; online **Rewrite Engine** applies them — **no LLM at query time**

Rewrite Engine (online)

- DuckDB parser → dialect-free logical IR, **9 operator types**
- **Fragment Index** pre-selects rules per node — sub-linear overhead
- Priority pass, then **MCTS/UCB1** over orderings; `ToSQL(dialect)`

Rule Manager

- Hash map: **fragment key** → **rule**, with priority
- Value-free shapes, e.g. `FILTER(JOIN(SCAN,SCAN))`
- **Hot-reload** — no engine restart

Rule Generator (offline)

- LLM rewrites a sample $\mathcal{Q}_S$; Distill extracts $(\phi, \tau, \mathcal{K})$, generalized past the instance
- Keys **deduplicate**; LLM emits a **C++17** `Rule` class, compiled and `dlopen`ed
- **3-phase verify**: smoke → SMT (EQ/NEQ/*unknown*) → sample test

ACKNOWLEDGEMENTS

Acknowledgements

Thank you to my committee members!



Carsten Binnig



Juergen Rilling



Abdelhak Bentaleb



Jun Yan

Acknowledgements

Thank you **Essam** and **Matthias**

for teaching me to ask better questions — and trusting me to answer them my own way.



Essam Mansour

Since September 2023



Matthias Boehm

Since April 2021

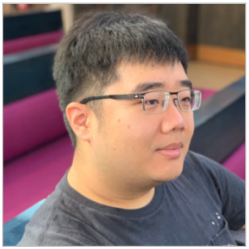
Acknowledgements

Thank you to the organizers of this defence!



Shin Hwei Tan

Chair



Tse-Hsun(Peter) Chen

Graduate Program Director